

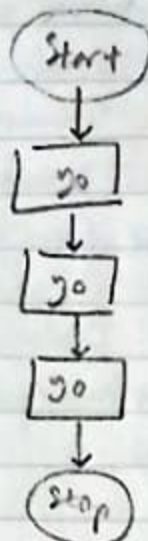
Loops

- The ability to do something again and again in a cycle

ex.

print("go")		go
print("go")		go
print("go")		go

flow of execution:



Instead of using the same function over again:

While

- an expression for a loop
- uses boolean expressions to evaluate a condition

ex. $i = 3$

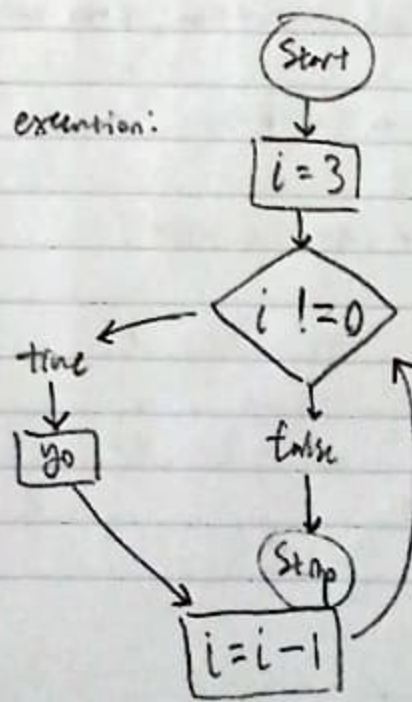
While $i \neq 0$:
 ... print("go") } this section will ALWAYS loop

Because i will remain 3 the whole time, "while" will always evaluate $i \neq 0$ to be true, therefore it will print "go" indefinitely.

IF an accidental loop ever occurs, use Ctrl C

$i = 3$
 While $i \neq 0$:
 ... print("go")
 ... $i = i - 1$

flow of execution:

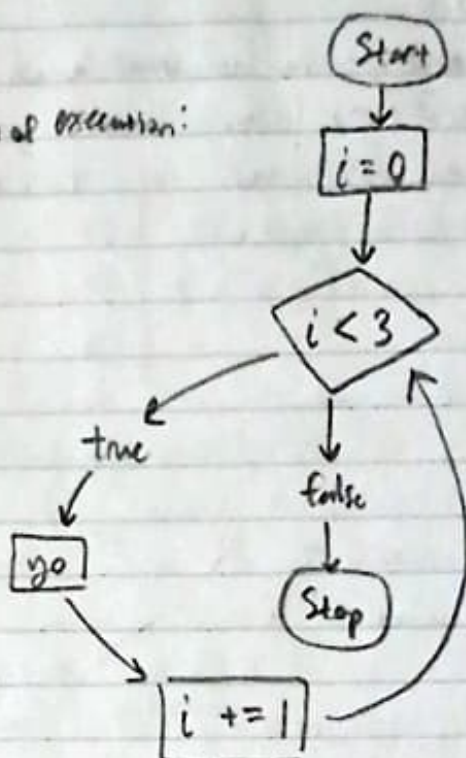


Start from 3, Subtract 1
 each time the loop is run
 until $i = 0$

Change the code slightly
 ex. `i = 0`
 `while i < 3:`
 `print("yo")`
 `i += 1`

yo
yo
yo

flow of execution:



Start from 0, add 1 each time
 loop is run until $i < 3$

another type of loop:

for

- utilizes lists in loops
- allows us to iterate a list of items

list

- another datatype, used to contain multiple values all in the same place

ex. `for i in [0, 1, 2]:`
 `print("yo")`

yo
yo
yo

`[0, 1, 2]` ← this is a list

to improve the list:

range

- returns a range of values

ex. `for i in range(3):`
 `print("yo")`

yo
yo
yo

a pythonic approach:

Even though the "i" variable is called in the "for" loop, I am not using it. If we use a variable just for the sake of the function and we don't really need it, use an underscore.

ex.

```
for _ in range(3):  
    print("yo")
```

yo
yo
yo

"looping" with multiplication operator to concatenate:

ex.

```
print("yo\n" * 3, end="")
```

function

argument

move to
next line

multiply by 3
(concatenate)

end the function with nothing

We can use loops to have the user input some correct value with conditions
While True: ← this is an infinite loop

... n = int(input("what's n?"))

... if n < 0:

... continue ← this continues the infinite loop

... else: as long as conditions are met

... break ← this breaks the infinite loop once the "else" condition is met

The below also works, but does not use "else" or "continue"

ex. While True:

... n = int(input("what's n?"))

... if n > 0:

... break

using different functions for the loop

ex. def main():

```
    num = get_num()
    gornan(num)
```

def get_num():

```
    while True:
```

```
        n = int(input("what's n?"))
```

```
        if n > 0:
```

```
            break
```

```
    return n
```

def gornan(n):

```
    for _ in range(n)
```

```
        print("go")
```

looking at lists:

ex. x = ["These", "are", "strings"] ← These strings are in a list

print(x[0])

← Variable [int] means the # int argument/value in a list is called through the print function.

This prints out the string "These".

utilizing the for loop

ex. x = ["These", "are", "strings"]

for word in x:

```
    print(word)
```

These

are

strings

There are more datatypes in python, which are for the most part "Iterable" meaning they are pieces of data that can be used in a loop.

If the for loop requires numbers to implement and iterate values:

ex. `xl = ["Hermione", "Harry", "Ron"]`
`for i in range(len(xl)):`
`... print(xl[i]):`

Hermione
Harry
Ron

len

- short for length
- function used to determine the length of a list

dict

- dictionaries
- a data structure that allows association of one value to another
- Keys and Values; words with definitions
- uses curly brackets in similar structure as a list

ex. `dogs = {` keys on the left/top, values on the right/bottom

`... "labrador": "loud"`
`... "chihuahua": "loud"`
`... "leo": "funky"`
`}`

labrador	chihuahua	leo
loud	loud	funky

this format shows how dict works

`print(dogs["leo"])`
`print(dogs["labrador"])`

To specify a certain key, we need to "index" into it.

funky
loud

Lists have numeric positions that are respective to the contents of that list, while Dictionaries allow us to use actual words.

utilizing the for loop:

ex. const. `for type in dogs:`
`... print(type, dogs[type])`

labrador	loud
chihuahua	loud
leo	funky

another representation of extra information

	type	personality	food
0	labrador	loud	meat
1	Chihuahua	loud	flesh
2	leo	funky	himself
3	Chungus	funky	

essentially, each type of dog is a dictionary. This table shows a dictionary that holds multiple dictionaries within it.

```
ex. dogs = [
    ... {"type": "labrador", "personality": "loud", "food": "meat"}
    ... {"type": "Chihuahua", "personality": "loud", "food": "flesh"}
    ... {"type": "leo", "personality": "funky", "food": "himself"}
    ... {"type": "Chungus", "personality": "funky", "food": None}
]
```

This is a special keyword that represents the absence of a

for bark in dogs:

```
... print(bark["type"], bark["personality"],
...       bark["food"], sep=" ", "
```

prints: labrador, loud, meat
 Chihuahua, loud, flesh
 leo, funky, himself
 Chungus, funky, None

functions allow users to create abstractions, simplifications or potentially complicated ideas

print() is basically just \n.